From Compression Methods to Deployment Decisions: Quality-Constrained Fine-Grained Strategy Optimization for Efficient LLM Inference

Anonymous submission

Abstract

Model compression has become an effective approach for speeding up large language model (LLM) inference. However, the availability of diverse compression methods does not directly translate into actionable deployment decisions: existing approaches lack a unified mechanism for selecting among compression techniques at a fine granularity—across model modules and across inference phases such as prefill and decoding—and they cannot automatically generate policies with appropriate quality–speed trade-offs for diverse deployment requirements. In this work, we propose **DASO** (Deployment-Aware Strategy Optimization), a deployment-oriented framework that turns a set of compression methods into fine-grained deployment decisions. DASO integrates quantization and pruning methods into a unified candidate space and automatically selects module-wise and phase-wise configurations under a user-defined quality constraint. To enable efficient optimization, we develop a hardware-aware, calibrated latency model, together with a quality model that estimates global quality degradation from local compression perturbations. A constrained optimizer then derives the latency-optimal policy within the given quality budget, which is deployed through an extended vLLM runtime. Experiments on two Llama models on RTX 5090, together with cross-platform validation on RTX PRO 6000, show that DASO achieves up to **2.26**× prefill and **1.55**× end-to-end speedups over BF16 while exposing budget-controlled quality–speed trade-offs.

Introduction

Large language models (LLMs) have demonstrated remarkable capabilities across a broad range of language understanding and generation tasks, including knowledge assessment, dialogue, text summarization, and machine translation (Touvron et al. 2023; Grattafiori et al. 2024). However, their continuously increasing parameter counts also incur substantial computation and memory-access overhead, which directly affects service latency, system throughput, and infrastructure costs in practical deployment (Kwon et al. 2023). Consequently, reducing inference overhead while satisfying model quality requirements has become a critical challenge.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

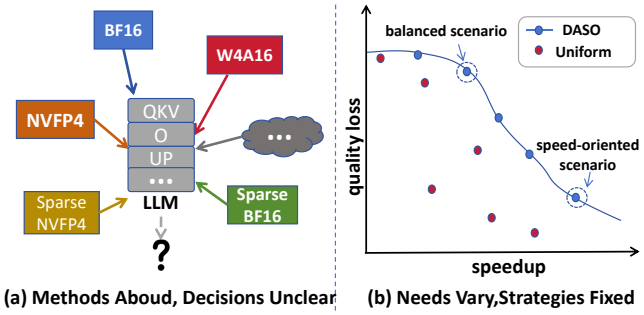


Figure 1: Motivation for deployment-aware strategy optimization. (a) Numerous compression methods are available, yet how to select among them remains unclear. (b) Diverse deployment needs cannot be met by fixed uniform policies, whereas DASO combines multiple methods at fine granularity to provide adaptable quality–speed trade-offs.

To reduce inference costs, extensive research has explored various post-training compression techniques, including quantization, pruning, and their combinations (Xiao et al. 2023; Frantar et al. 2023; Lin et al. 2024; Frantar and Alistarh 2023; Sun et al. 2024). These methods compress models by reducing parameter precision or pruning redundant weights. For example, SparseGPT enables one-shot pruning of LLMs using approximate second-order information (Frantar and Alistarh 2023); AWQ preserves salient weights based on activation statistics to improve the accuracy of low-bit weight-only quantization (Lin et al. 2024); SmoothQuant smooths the numerical distributions of weights and activations to support efficient W8A8 inference (Xiao et al. 2023); and the recent NVFP4 format further leverages native low-precision hardware to improve the efficiency of 4-bit computation (Alvarez et al. 2025). Collectively, these studies have substantially expanded the design space of compression and inference acceleration techniques for LLMs.

However, from the perspective of practical deployment, the availability of diverse compression techniques does not directly translate into actionable deployment decisions. This gap manifests in two ways, as illustrated in Figure 1. First, although individual methods have each shown advantages in specific settings, practitioners lack a unified mechanism for selecting among them: given a target model, hardware

platform, and workload, which compression method—or combination—should be used, and at what granularity? Second, different serving scenarios impose different requirements on model quality and inference performance. Some services require the original model quality to be preserved to a greater extent, whereas others can tolerate a certain degree of quality degradation in exchange for acceleration gains. Yet existing approaches typically provide only a fixed configuration or a small set of predefined options, making it difficult to deliver controllable quality–speed trade-offs for diverse deployment requirements. Thus, while individual compression methods have each demonstrated effectiveness, the central challenge in practical deployment has shifted to **how to systematically select and configure them into an optimal deployment policy under a given quality requirement**.

To address this challenge, we propose **DASO**, a deployment-oriented framework that translates diverse compression methods into fine-grained deployment decisions. The framework incorporates multiple representative compression methods into a unified candidate space and supports fine-grained module-wise and inference-phase-wise configurations. To enable unified comparison among candidate policies and provide controllable trade-offs, we develop a latency model tailored to the target hardware and inference workload, together with a quality model that estimates the resulting model quality degradation. We then formulate policy generation as an inference latency minimization problem subject to a model quality constraint. This formulation enables model deployers to automatically generate optimized compression policies according to the requirements of specific serving scenarios. We further integrate the proposed framework into a real-world LLM inference system and evaluate it across multiple models, workloads, and quality constraints. The results demonstrate that our framework achieves more favorable end-to-end quality–speed trade-offs than uniform whole-model compression configurations.

Our main contributions are summarized as follows:

- We propose a deployment-oriented framework that unifies multiple compression methods into a common policy space with heterogeneous module-wise and phase-wise configurations, which can minimize inference latency under a model quality constraint to deliver controllable quality–speed trade-offs.
- We develop a hardware-aware latency model and a model-aware quality model for heterogeneous compression policies. These models characterize the latency benefits and quality costs of different compression methods with respect to layer characteristics, inference phases, and practical workloads, enabling efficient evaluation of a large policy space and providing reliable estimates for compression policy optimization.
- We integrate the proposed framework into a real-world LLM inference system and conduct extensive experiments across multiple models, inference workloads, and quality budgets. The results show that our framework consistently achieves better end-to-end quality–speed trade-offs than uniform whole-model compression schemes.

Related Work

Quantization and Pruning for LLMs. Post-training compression of large language models mainly focuses on quantization and pruning. Weight–activation quantization reduces memory overhead and leverages hardware low-precision units for acceleration, but must address activation outliers. SmoothQuant transfers quantization difficulty from activations to weights, while QuaRot and FlatQuant suppress outliers through fixed or learnable transformations (Xiao et al. 2023; Ashkboos et al. 2024b; Sun et al. 2025). Weight-only methods such as GPTQ and AWQ reduce weight-loading overhead and are particularly suitable for memory-bound decoding, while Marlin provides efficient W4A16 kernels for further practical acceleration (Frantar et al. 2023; Lin et al. 2024; Frantar et al. 2025). Recently, hardware-supported FP4 formats, particularly NVFP4, have provided a promising path toward low-precision inference (Alvarez et al. 2025). For pruning, SparseGPT and Wanda remove redundant weights with limited calibration cost, whereas SliceGPT removes structured dimensions to produce smaller dense models (Frantar and Alistarh 2023; Sun et al. 2024; Ashkboos et al. 2024a). Quantization and pruning can also be combined to enable sparse low-precision execution (Guo et al. 2024).

Fine-Grained Compression Configuration. Several existing studies allocate bit widths or sparsity ratios across layers using sensitivity or reconstruction objectives (Guan et al. 2024; Huang et al. 2025; Yin et al. 2024; Xu et al. 2024), while PMPD and FlexQuant vary precision across phases or tokens (Chen et al. 2025; Liu et al. 2025). LLM-PQ further extends this idea to heterogeneous GPU clusters (Zhao et al. 2024). These fine-grained allocation techniques operate at a different design level from DASO's cross-method selection and could be incorporated as additional candidate configurations; we leave this integration to future work.

Method

Framework Overview and Problem Formulation

We consider the problem of compression policy selection for practical large language model deployment. Given a target model, hardware platform, inference workload, and user-specified quality-loss budget, the framework selects a deployment policy from a unified set of candidate methods, including quantization, pruning, and their combinations. The objective is to minimize inference latency while satisfying the quality constraint. Rather than experimenting with numerous methods, we place multiple compression methods in a common candidate space and formulate policy generation as a deployment-specific constrained optimization problem.

Figure 2 overviews DASO. As shown in Fig. 2(a), the latency model calibrates a method-aware analytical baseline against measurements through complementary global and local predictors, while the quality model combines local output perturbations with structure-aware sensitivity and a small set of policy-level calibration measurements. Together, the two models assign comparable latency and quality costs to every legal action for each configurable module and inference phase. Figure 2(b) shows how these costs are used at deployment time: given the model, hardware, workload, candidate

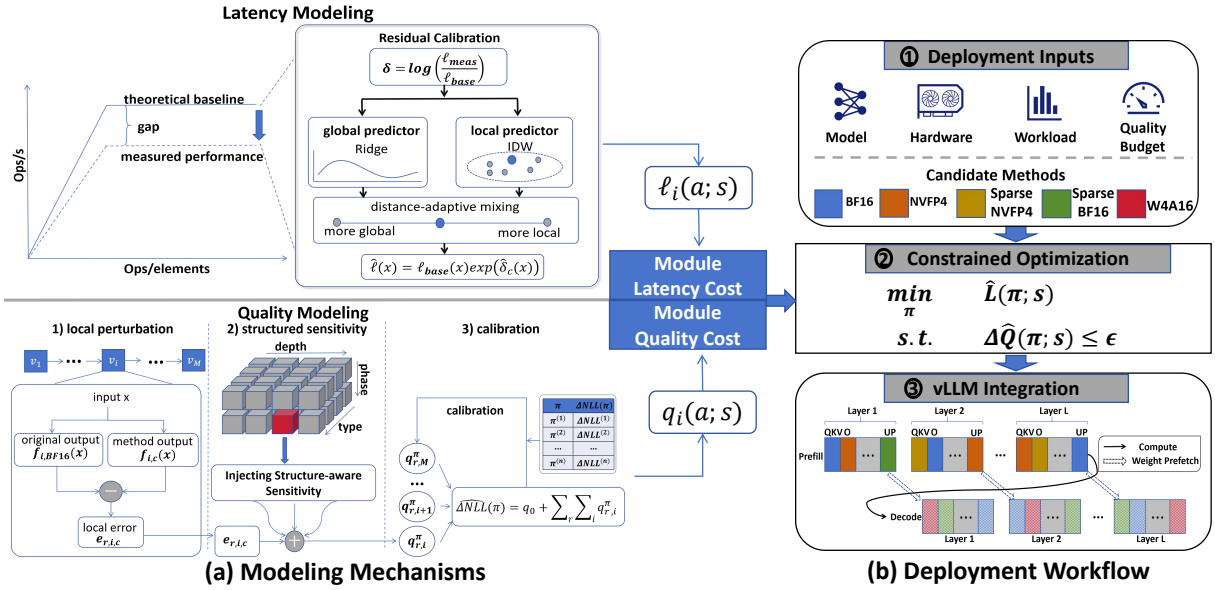


Figure 2: Overview of **DASO**. (a) The latency and quality models estimate the cost of each compression choice for different modules and inference phases. (b) Given the deployment setting, candidate methods, and quality budget, DASO selects a module-wise and phase-wise compression policy and executes it in the extended vLLM runtime.

methods, and quality budget, the constrained optimizer selects a module-wise, phase-specific policy for prefill and decoding, and the extended vLLM runtime executes the result while overlapping decoding-weight prefetch with prefill computation. This fine granularity allows the optimizer to account for shape- and phase-dependent latency gains as well as heterogeneous module sensitivity, leading to the policy definition below.

Let the model contain M configurable modules $\mathcal{V} = \{v_i\}_{i=1}^{M}$. Given a set of candidate compression methods $\mathcal{C}$, the action of module v_i is the phase pair $a_i = (a_i^p, a_i^d)$, where $a_i^p, a_i^d \in \mathcal{C}$ denote the methods used during prefill and decoding. The actions of all modules jointly define a complete policy $\pi = (a_1, \ldots, a_M)$. A deployment scenario $s = (B, S, T)$ specifies the batch size, input sequence length, and generation length.

Let $\mathcal{A}_i \subseteq \mathcal{C} \times \mathcal{C}$ be the legal phase-pair actions for module v_i after removing configurations unsupported by the target hardware and backend, with $a_i \in \mathcal{A}_i$ for $i = 1, \ldots, M$. Given a quality-loss budget ϵ, heterogeneous compression policy generation is formulated as

$$\min_{\pi} \ \widehat{L}(\pi; s) \qquad \text{s.t.} \qquad \widehat{\Delta Q}(\pi; s) \le \epsilon. \tag{1}$$

where $\widehat{L}(\pi; s)$ and $\widehat{\Delta Q}(\pi; s)$ denote the predicted inference latency and quality degradation of policy π under scenario s. By varying ϵ, the framework produces policies with different quality–speed trade-offs. Since the policy space grows combinatorially with M, exhaustively evaluating every candidate is infeasible. The following sections introduce the latency and quality models for efficient cost estimation, followed by the optimization procedure and runtime deployment.

Latency Modeling

To optimize heterogeneous compression policies, DASO requires a comparable latency cost for every legal module–action pair under a given workload shape. This cost must reflect the realized execution time of the corresponding compressed Linear module, which cannot be reliably inferred from nominal indicators such as bit width, compression ratio, or FLOPs due to two challenges: (i) different compression methods involve distinct execution pipelines, and (ii) analytical cost models often deviate systematically from measured latency.

To address the first challenge, we develop a method-aware Roofline baseline that places different compression methods in a unified latency formulation while preserving their distinct execution characteristics. For a candidate method $c \in \mathcal{C}$, we use the GEMM convention $Y = XW^\top$, where the input X, weight W, and output Y have shapes $m \times k$, $n \times k$, and $m \times n$, respectively. The baseline latency is defined as

$$\ell_c^{\text{base}}(m, n, k) = \max\left(\frac{F_c(m, n, k)}{P_c}, \frac{D_c(m, n, k)}{W_c}\right) + \ell_c^0. \tag{2}$$

Here, $F_c(m, n, k)$ and $D_c(m, n, k)$ denote the effective computation and data movement of method c, respectively. P_c and W_c denote the effective compute throughput and memory bandwidth of the corresponding kernel on the target platform, and ℓ_c^0 captures other fixed overheads such as kernel launch.

This formulation builds on the classical Roofline model (Williams, Waterman, and Patterson 2009), which decomposes the dominant execution cost into computation and data movement, and we use it to estimate latency lower bounds for different compression methods. We instantiate this formulation by defining method-specific F_c and D_c,

Method	F_c	D_c
Dense BF16	$2mnk$	$2mk + 2nk + 2mn$
Sparse BF16	mnk	$2mk + \frac{9}{8}nk + 2mn$
Dense NVFP4	$2mnk$	$2.5mk + 0.5nk + \frac{mk+nk}{16} + 2mn$
Sparse NVFP4	mnk	$2.5mk + \frac{mk}{32} + \frac{11}{32}nk + 2mn$
W4A16 Marlin	$2mnk$	$2mk + 0.5nk + \frac{nk}{16} + 2mn$

Table 1: Method-specific effective computation F_c and data movement D_c used by the analytical latency baseline for a Linear layer with matrix-multiplication shape (m, n, k).

which account for differences in numerical representation, activation processing, scale access, and sparse metadata. Table 1 summarizes these definitions, with detailed derivations provided in Appendix. Consequently, the baseline preserves the physical scaling of computation and data movement across matrix shapes, providing generalizable estimates even for shapes absent from the profiling set.

The method-aware Roofline baseline captures the overall compute–memory trend, but it cannot fully represent shape-dependent kernel efficiency on the target platform. To address the second challenge, rather than directly regressing absolute latency, we learn the logarithmic multiplicative residual between the measured latency and the Roofline baseline. For a profiled shape $x_j = (m_j, n_j, k_j)$ under method c, the residual target is

$$\delta_{c,j} = \log \frac{\ell_{c,j}^{\text{meas}}}{\ell_c^{\text{base}}(x_j)}, \tag{3}$$

which represents the relative deviation not explained by the analytical model.

The residual predictor combines global and local shape information. A regularized ridge regressor learns a smooth global trend from matrix dimensions and aspect ratios, together with tile-count and tail-ratio statistics. In parallel, a local predictor interpolates the residuals of nearby profiled shapes using inverse-distance weights, preserving kernel behaviors that are difficult to express with a single global function. We adaptively combine the two predictions according to the distance $d_{\min}(x)$ between the target shape x and its nearest profiled shape:

$$\widehat{\delta}_c(x) = \lambda(d_{\min}(x))\, \widehat{\delta}_c^{\text{local}}(x) + [1 - \lambda(d_{\min}(x))]\, \widehat{\delta}_c^{\text{global}}(x), \tag{4}$$

where $\lambda(\cdot)$ decreases with distance, assigning more weight to local interpolation near profiled shapes and more weight to the regularized global model when extrapolating farther away. The final latency estimate is

$$\widehat{\ell}_c(x) = \ell_c^{\text{base}}(x) \exp\left(\widehat{\delta}_c(x)\right). \tag{5}$$

Modeling a logarithmic residual captures relative errors across kernels with different latency scales, while exponentiation yields a positive multiplicative correction. Detailed feature definitions, distance normalization, local interpolation, and the mixing function are provided in Appendix.

Quality Modeling

The quality model assigns a quality cost to each compression action on each configurable module, supplying the penalty

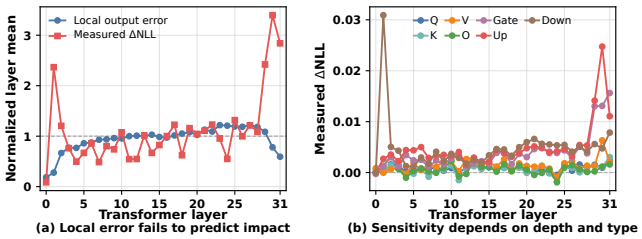


Figure 3: (a) Local output error poorly reflects global quality impact. (b) Sensitivity varies substantially across model depth and Linear projection types.

terms needed by the constrained optimizer. Building these per-module costs raises two questions: (i) what signal should serve as the quality measure, and (ii) how to estimate the quality impact of compressing a single module such that the individual costs remain informative when many modules are compressed jointly.

For the first question, directly using downstream metrics such as accuracy or BLEU (Papineni et al. 2002) incurs substantial evaluation cost and depends heavily on dataset and evaluation protocol, making them unsuitable as a stable and efficient optimization signal. We therefore use teacher-forced negative log-likelihood (NLL) as the quality measure. Given a reference sequence $\mathbf{y}$, the NLL of policy π is

$$\text{NLL}(\pi) = -\frac{1}{|\mathbf{y}|} \sum_{t=1}^{|\mathbf{y}|} \log p_\pi(y_t \mid y_{<t}). \tag{6}$$

We quantify compression-induced degradation by the increase over the BF16 model, $\Delta\text{NLL}(\pi) = \text{NLL}(\pi) - \text{NLL}(\pi_{\text{BF16}})$.

For the second question, a direct approach is to compress each module independently and measure its ΔNLL impact. However, this requires hundreds of full-model evaluations, making it impractical. We instead construct module-level costs in two steps. First, we characterize each candidate method by an inexpensive local perturbation that can be evaluated from cached module inputs without repeatedly running the full model. Let $f_{i,c}$ denote the function implemented by module v_i under method $c \in \mathcal{C}$, with $f_{i,\text{BF16}}$ denoting its BF16 baseline, and let $\mathcal{X}_{r,i}$ be the set of input activations collected for this module during inference phase r. We define the local perturbation as

$$e_{r,i,c} = \frac{\sum_{x \in \mathcal{X}_{r,i}} \|f_{i,c}(x) - f_{i,\text{BF16}}(x)\|_2^2}{\max\left\{\sum_{x \in \mathcal{X}_{r,i}} \|f_{i,\text{BF16}}(x)\|_2^2, \eta\right\}}. \tag{7}$$

Here, $\eta = 10^{-12}$ is a stabilizing threshold. By measuring the normalized output perturbation under identical module inputs, this characterization provides a unified local metric across different compression methods.

The local error $e_{r,i,c}$ captures perturbation magnitude, but cannot directly predict model-level ΔNLL: as Figure 3 shows, the same local error can have markedly different consequences depending on the compression method, module type, depth, and inference phase. This observation directly

motivates our modeling choice—the second step learns a structured mapping that decomposes sensitivity along exactly these dimensions:

$$q_{r,i,c} = e_{r,i,c} \frac{\mathrm{ReLU}\left(g + \rho_r + u_c + \beta_{b_i} + \tau_{t_i}\right)}{s_{\mathrm{feat}}}. \quad (8)$$

Here, b_i and t_i denote the depth-region and module-type labels of module v_i, respectively. The terms g, ρ_r, u_c, β_{b_i}, and τ_{t_i} capture the shared global, inference-phase, compression-method, depth-region, and module-type effects, respectively. The constant $s_{\mathrm{feat}} > 0$ is the average total mass of the structured local-perturbation features over the training policies and is used only for numerical normalization. Modules in the same depth region or of the same type therefore share the corresponding sensitivity parameters. This factored design pools information across structurally similar modules and greatly reduces the parameter count, allowing the model to be fitted from only a small set of complete policies. The ReLU enforces $q_{r,i,c} \geq 0$, ensuring compression costs are non-negative and preventing quality contributions from spuriously canceling across modules.

For a policy π, let $q^{\pi}_{r,i} \equiv q_{r,i,a^r_i}$ denote the quality cost of its selected method for module v_i in phase r. We define the predicted policy-level quality degradation as the sum of these selected costs:

$$\widehat{\Delta\mathrm{NLL}}(\pi) = q_0 + \sum_{r\in\mathcal{R}} \sum_{i=1}^{M} q^{\pi}_{r,i}, \quad (9)$$

where q_0 is the fitted regression intercept. The phase set is $\mathcal{R} = \{p\}$ for prefill-only inference and $\mathcal{R} = \{p, d\}$ for prefill–decode inference. We fit the shared parameters by minimizing the regularized mean-squared error between $\widehat{\Delta\mathrm{NLL}}(\pi)$ and the measured $\Delta\mathrm{NLL}(\pi)$ of a small set of complete heterogeneous policies.

This additive form is designed to match the knapsack solver: each $q_{r,i,c}$ serves both as a term in the quality prediction and as an independent cost in the quality budget constraint (Eq. 11). Moreover, because fitting uses complete policies rather than isolated modules, the learned costs implicitly absorb joint compression effects, so their sum remains predictive even when many modules are compressed together.

Strategy Optimization and Deployment

Strategy Optimization. For a deployment scenario $s = (B, S, T)$ and a module v_i with weight shape (n_i, k_i), let $a = (a^p, a^d) \in \mathcal{A}_i$ be a legal phase-pair action. Prefill processes BS tokens once, whereas decoding processes B tokens in each of the remaining $T - 1$ generation steps. The resulting latency cost of assigning a to module v_i is

$$\ell_i(a; s) = \widehat{\ell}_{a^p}(BS, n_i, k_i) + (T - 1)\widehat{\ell}_{a^d}(B, n_i, k_i), \quad (10)$$

where $\widehat{\ell}_c(m, n, k)$ is the predicted latency under method c for matrix shape (m, n, k) obtained from the calibrated latency model. For a prefill-only scenario, only the first term is retained. Accordingly, the policy latency estimate used in Eq. 1 is $\widehat{L}(\pi; s) = \sum_{i=1}^{M} \ell_i(a_i; s)$. From Eq. 9, the corresponding quality cost is $q_i(a; s) = \sum_{r\in\mathcal{R}} q_{r,i,a^r}$. Thus, every legal module–action pair is represented by an additive latency–quality cost pair $\big(\ell_i(a; s), q_i(a; s)\big)$.

The abstract problem in Eq. 1 therefore reduces to a multiple-choice knapsack. Let $z_{i,a} \in \{0, 1\}$ indicate whether action $a \in \mathcal{A}_i$ is selected for module v_i, with $\sum_{a\in\mathcal{A}_i} z_{i,a} = 1$ for every module i. The optimization is

$$\min_{z} \sum_{i=1}^{M} \sum_{a\in\mathcal{A}_i} z_{i,a}\ell_i(a; s) \text{ s.t. } q_0 + \sum_{i=1}^{M} \sum_{a\in\mathcal{A}_i} z_{i,a} q_i(a; s) \leq \epsilon. \quad (11)$$

The intercept q_0 is policy-independent and can equivalently be subtracted once from the quality budget. We discretize the remaining budget into H bins and solve the problem by dynamic programming, maintaining the minimum latency attainable at each cumulative quality cost while processing modules sequentially. The time complexity is $O(H \sum_i |\mathcal{A}_i|)$ with $O(H)$ working memory. Sweeping ϵ yields candidate policies along the predicted quality–speed frontier. In our implementation, the solver generates 25 Pareto-optimal policies for Llama-2-7B-Chat in a few seconds.

Runtime Deployment. We extend vLLM with phase-aware backend dispatch so that each Linear module executes a^p_i during prefill and a^d_i during decoding. The main systems challenge is the resulting representation switch between phases: a synchronous implementation materializes all decoding weights at the phase boundary, creating a global stall. DASO instead adopts a layer-wise asynchronous materialization pipeline. Once a layer completes prefill, its decoding representation is staged on a dedicated copy stream while the compute stream continues prefill on subsequent layers. This overlap moves most phase-switching work off the critical path, making fine-grained phase-heterogeneous policies practical for end-to-end serving.

Experiments

We organize the evaluation around five key questions:

- **Q1. Single-method speed ceiling.** Can DASO exceed the speedup of every uniform method?
- **Q2. Quality retention.** Can DASO achieve practical speedups under a tight quality budget?
- **Q3. Quality–speed control.** Can DASO provide operating points for different quality budgets?
- **Q4. Hardware adaptability.** Can DASO adapt its policies to different GPU platforms?
- **Q5. Ablation study.** Do the proposed modeling and runtime components improve estimation and deployment?

Experimental Setup

Models and workloads. We evaluate Llama-2-7B-Chat and Llama-3.1-8B-Instruct on RTX 5090 and additionally on RTX PRO 6000 for cross-platform validation. We consider two representative workloads, both with batch size $B = 8$ and input length $S = 2048$: prefill-only inference and prefill–decode inference with generation length $T = 64$. Prefill quality is accuracy on ARC-Challenge, ARC-Easy,

Model	Method	Prefill-only					Prefill–Decode					
		ARC-C	ARC-E	Wino.	MMLU	TTFT	CNN/DM	DialogSum	IWSLT'17	TTFT	TPOT	E2E
Llama-2 7B-Chat	BF16	43.34	72.81	68.27	46.57	1.00×	23.67	21.69	19.30	1.00×	1.00×	1.00×
	Dense NVFP4	43.17	71.13	67.80	44.94	1.71×	**24.27**	20.59	16.84	**1.64×**	0.55×	0.80×
	W4A16 Marlin	43.09	**71.89**	**68.82**	**45.66**	1.02×	23.73	**21.77**	18.85	0.97×	1.50×	1.19×
	Sparse BF16	35.07	63.85	63.85	28.24	1.53×	15.35	13.54	3.90	1.56×	0.91×	1.13×
	Sparse NVFP4	23.38	36.62	50.43	23.09	1.86×	15.96	13.57	1.57	1.62×	0.52×	0.77×
	DASO (Max speed)	27.73	53.16	56.75	23.51	**2.08×**	18.56	13.57	2.60	1.50×	**1.60×**	**1.55×**
	DASO (Balanced)	**43.69**	70.58	67.32	44.35	1.74×	23.80	21.40	**21.08**	1.03×	1.26×	1.14×
Llama-3.1 8B-Instruct	BF16	55.63	81.99	73.48	68.40	1.00×	19.39	13.48	10.78	1.00×	1.00×	1.00×
	Dense NVFP4	51.11	79.29	72.06	63.50	1.76×	17.11	9.54	10.51	1.71×	0.53×	0.83×
	W4A16 Marlin	**53.67**	**80.93**	73.09	66.20	0.98×	17.50	9.75	**10.56**	0.98×	**1.27×**	1.10×
	Sparse BF16	38.74	68.18	66.06	42.96	1.54×	6.76	5.11	3.04	1.53×	0.88×	1.13×
	Sparse NVFP4	26.37	45.66	53.28	23.88	1.93×	4.25	0.41	0.05	**1.81×**	0.49×	0.79×
	DASO (Max speed)	33.02	61.28	61.09	31.14	**2.26×**	14.59	4.21	1.29	1.74×	1.18×	**1.42×**
	DASO (Balanced)	51.45	79.08	**73.80**	**66.50**	1.59×	**18.84**	**11.48**	10.55	1.40×	1.13×	1.26×

Table 2: Measured downstream quality and speedups of uniform baselines and DASO on RTX 5090. Time to first token (TTFT), time per output token (TPOT), and end-to-end (E2E) speedups are relative to BF16 under the same workload. Excluding the BF16 reference, darker shading indicates a higher rank among the top three results in each column. "Balanced" and "Max speed" are returned under tight and relaxed quality budgets, respectively.

WinoGrande, and MMLU (Clark et al. 2018; Sakaguchi et al. 2020; Hendrycks et al. 2021); generation quality is ROUGE-L on CNN/DailyMail and DialogSum (Nallapati et al. 2016; Chen et al. 2021; Lin 2004), and SacreBLEU (Post 2018) on IWSLT 2017 (Cettolo et al. 2017). We report TTFT, TPOT, and E2E speedups over BF16.

Baselines and system. We use five widely adopted compression choices with optimized kernel support as uniform baselines: BF16, dense NVFP4 (Alvarez et al. 2025), Sparse BF16 (Frantar and Alistarh 2023), sparse NVFP4, and W4A16 Marlin (Frantar et al. 2025). As DASO targets realized serving performance, we select these compression methods with mature, efficient kernel implementations rather than per-layer allocation schemes. Both sparse baselines use 50% 2:4 semi-structured sparsity, with pruning masks calibrated by SparseGPT. Dense and sparse NVFP4 use CUTLASS kernels (Thakkar et al. 2023), sparse BF16 uses cuSPARSELt (Busato and Pool 2020), and W4A16 Marlin uses the Marlin kernel. We implement DASO on top of vLLM 0.11.1 (Kwon et al. 2023) and conduct all experiments on NVIDIA RTX 5090 and RTX PRO 6000 GPUs, profiling each GPU separately and measuring all policies under the same runtime protocol for a fair comparison.

Single-Method Speed Ceiling (Q1)

Tables 2 and 3 show that DASO consistently breaks the single-method speed ceiling across all model–platform–workload settings. Its *Max speed* policies achieve 1.49×–2.26× prefill speedups and 1.42×–1.55× generation E2E speedups, surpassing every uniform baseline. These results reflect that no single method is universally effective: on RTX 5090, dense NVFP4 strongly accelerates prefill but slows decoding, whereas W4A16 Marlin provides little prefill gain while reaching up to 1.50× TPOT speedup. Unlike uniform methods, DASO selects fine-grained module-wise and phase-wise compression configurations for each deployment scenario, overcoming the single-method speed ceiling.

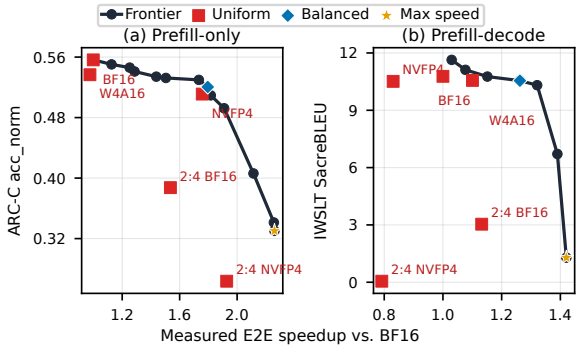


Figure 4: Budget-controlled policies for Llama-3.1-8B-Instruct on RTX 5090. Dark circles form the measured DASO frontier; red squares are uniform baselines. Diamonds and stars mark the *Balanced* and *Max speed* policies.

Quality Retention at Practical Speedups (Q2)

To answer Q2, we report one illustrative *Balanced* policy selected under a tight quality budget for each setting. As highlighted in Tables 2 and 3, the Balanced policies consistently rank among the top three non-BF16 configurations—and often first—across the reported quality metrics while still delivering considerable practical speedups. For Llama-3.1-8B-Instruct generation on RTX 5090, *Balanced* reaches 1.26× E2E speedup, compared with 1.10× for W4A16 Marlin, while achieving higher CNN/DailyMail and DialogSum ROUGE-L and essentially matching IWSLT. On RTX PRO 6000, the *Balanced* policy reaches 1.25× prefill speedup with 56.91 average accuracy, compared with 1.31× and 47.06 for the fastest uniform Sparse BF16.

Method	Prefill-only		Prefill–Decode			
	Acc.	TTFT	Score	TTFT	TPOT	E2E
BF16	57.69	1.00×	21.53	1.00×	1.00×	1.00×
Dense NVFP4	56.76	1.08×	20.77	1.07×	0.71×	0.81×
W4A16 Marlin	**57.25**	0.90×	**21.57**	0.90×	1.52×	1.21×
Sparse BF16	47.06	1.31×	12.34	1.31×	1.08×	1.16×
Sparse NVFP4	34.59	1.19×	11.84	1.18×	0.68×	0.81×
DASO (Max speed)	40.10	**1.49**×	12.45	**1.33**×	**1.53**×	**1.45**×
DASO (Balanced)	56.91	1.25×	19.13	1.16×	1.44×	1.32×

Table 3: Results for Llama-2-7B-Chat on RTX PRO 6000.

Controllable Quality–Speed Trade-offs (Q3)

We evaluate DASO across a range of ΔNLL budgets. As shown in Figure 4, DASO produces a controllable quality–speed frontier for both workloads. Tight budgets keep DASO in the high-quality region, while relaxing the budget progressively increases speed, extending to 2.26× for prefill and 1.42× E2E for prefill–decode. More importantly, the DASO frontier forms an upper-right envelope over nearly all uniform baselines in both panels. Uniform methods offer only isolated operating points, and a method that performs well in one inference phase or workload can be inefficient or overly destructive in another. In contrast, DASO jointly selects module-wise and phase-wise compression actions to construct an optimized policy for each deployment scenario and quality budget, fully exploiting the available compression potential and enabling flexible quality–speed trade-offs.

Adaptation Across Hardware Platforms (Q4)

Hardware characteristics substantially change the relative benefits of individual compression methods. As shown in Table 3, the fastest uniform prefill method for Llama-2-7B-Chat changes from sparse NVFP4 on RTX 5090 (1.86×) to sparse BF16 on RTX PRO 6000 (1.31×). For compactness, Acc. reports the average of the four prefill accuracy metrics, while Score reports the average of the three generation metrics. DASO adapts its fine-grained policy to the new platform, reaching 1.49× prefill and 1.45× E2E speedups, beyond the corresponding uniform ceilings of 1.31× and 1.21×. These results show that platform-specific optimization allows DASO to preserve its advantage despite changes in kernel behavior and hardware balance.

Ablation Study (Q5)

We conduct component-wise ablations from cost modeling to runtime deployment on Llama-3.1-8B-Instruct. As shown in Table 4, the method-specific Roofline prior and shape correction reduce unseen-shape kernel-latency mean absolute percentage error (MAPE) from 14.80% to 8.01%. For quality modeling, structure and phase awareness reduce the held-out ΔNLL mean absolute error (MAE) from 0.0571 to 0.0413. At deployment, asynchronous phase switching overlaps 99.46% of the transfer work with useful computation, reducing the median vLLM E2E latency of the *Max speed* policy from 1625.87 to 1521.73 ms, a 6.41% reduction. These results show that the proposed designs improve both cost estimation and realized serving performance.

Ablation type	Variant	Result
Kernel modeling	Direct fit	14.80%
	+ Roofline	10.78%
	+ Shape corr.	**8.01%**
Quality modeling	Local aggregation	0.0571
	+ Structure-aware	0.0468
	+ Phase-aware	**0.0413**
Runtime deployment	Cumulative transfer	354.94 ms
	Exposed after overlap	**1.92 ms** (99.46%)

Table 4: Ablation from cost modeling to runtime deployment. Kernel modeling reports MAPE, quality modeling reports ΔNLL MAE, and runtime deployment reports phase-switch transfer work and its exposed portion after overlap.

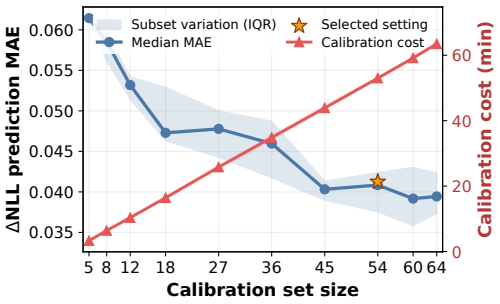


Figure 5: Sensitivity to the number of quality-calibration policies. The line and shading show held-out ΔNLL MAE, the secondary axis reports labeling cost, and the star marks the main configuration.

We further analyze sensitivity to the calibration-set size in Figure 5. Increasing the number of policy-level NLL labels from 5 to 45 reduces the median held-out MAE from 0.0614 to 0.0403, after which the improvement becomes modest while labeling cost continues to grow. We therefore use 54 calibration policies, achieving 0.0413 MAE at a cost of 48.6 GPU-minutes to balance modeling accuracy and calibration overhead.

Conclusion

We presented **DASO**, a deployment-oriented framework that bridges the gap between the availability of diverse compression methods and the need for actionable, quality-constrained deployment decisions under real hardware and workload constraints. Rather than applying one compression format uniformly, DASO jointly selects module-wise and phase-wise actions under a quality constraint, with calibrated latency and quality models enabling efficient search and an extended vLLM runtime supporting deployment. Across Llama models with cross-platform validation, DASO consistently improves quality–speed trade-offs over uniform baselines, demonstrating the effectiveness of our approach. Future work includes extending DASO to support finer-grained budget allocation schemes and more dynamic serving scenarios.

References

Alvarez, E.; Almog, O.; Chung, E.; Layton, S.; Stosic, D.; Krashinsky, R.; and Aubrey, K. 2025. Introducing NVFP4 for Efficient and Accurate Low-Precision Inference. NVIDIA Technical Blog. https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/. Accessed: 2026-07-28.

Ashkboos, S.; Croci, M. L.; do Nascimento, M. G.; Hoefler, T.; and Hensman, J. 2024a. SliceGPT: Compress Large Language Models by Deleting Rows and Columns. In *International Conference on Learning Representations*.

Ashkboos, S.; Mohtashami, A.; Croci, M. L.; Li, B.; Cameron, P.; Jaggi, M.; Alistarh, D.; Hoefler, T.; and Hensman, J. 2024b. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. In *Advances in Neural Information Processing Systems*, volume 37, 100213–100240.

Busato, F.; and Pool, J. 2020. Exploiting NVIDIA Ampere Structured Sparsity with cuSPARSELt. https://developer.nvidia.com/blog/exploiting-ampere-structured-sparsity-with-cusparselt/. Accessed: 2026-07-28.

Cettolo, M.; Federico, M.; Bentivogli, L.; Niehues, J.; Stüker, S.; Sudoh, K.; Yoshino, K.; and Federmann, C. 2017. Overview of the IWSLT 2017 Evaluation Campaign. In *Proceedings of the 14th International Conference on Spoken Language Translation*, 2–14. International Workshop on Spoken Language Translation.

Chen, H. M.; Tan, F.; Kouris, A.; Lee, R.; Fan, H.; and Venieris, S. I. 2025. Progressive Mixed-Precision Decoding for Efficient LLM Inference. In *International Conference on Learning Representations*.

Chen, Y.; Liu, Y.; Chen, L.; and Zhang, Y. 2021. DialogSum: A Real-Life Scenario Dialogue Summarization Dataset. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 5062–5074. Association for Computational Linguistics.

Clark, P.; Cowhey, I.; Etzioni, O.; Khot, T.; Sabharwal, A.; Schoenick, C.; and Tafjord, O. 2018. Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457*.

Frantar, E.; and Alistarh, D. 2023. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, 10323–10337. PMLR.

Frantar, E.; Ashkboos, S.; Hoefler, T.; and Alistarh, D. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers. In *International Conference on Learning Representations*.

Frantar, E.; Castro, R. L.; Chen, J.; Hoefler, T.; and Alistarh, D. 2025. MARLIN: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 239–251. Association for Computing Machinery.

Grattafiori, A.; et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.

Guan, Z.; Huang, H.; Su, Y.; Huang, H.; Wong, N.; and Yu, H. 2024. APTQ: Attention-Aware Post-Training Mixed-Precision Quantization for Large Language Models. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 1–6. Association for Computing Machinery.

Guo, J.; Wu, J.; Wang, Z.; Liu, J.; Yang, G.; Ding, Y.; Gong, R.; Qin, H.; and Liu, X. 2024. Compressing Large Language Models by Joint Sparsification and Quantization. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 16945–16957. PMLR.

Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; and Steinhardt, J. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.

Huang, W.; Qin, H.; Liu, Y.; Li, Y.; Liu, Q.; Liu, X.; Benini, L.; Magno, M.; Zhang, S.; and Qi, X. 2025. SliM-LLM: Salience-Driven Mixed-Precision Quantization for Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 25672–25692. PMLR.

Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; Gonzalez, J. E.; Zhang, H.; and Stoica, I. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626. Association for Computing Machinery.

Lin, C.-Y. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*, 74–81. Association for Computational Linguistics.

Lin, J.; Tang, J.; Tang, H.; Yang, S.; Chen, W.-M.; Wang, W.-C.; Xiao, G.; Dang, X.; Gan, C.; and Han, S. 2024. AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems*, volume 6.

Liu, F.; Wang, Z.; Xia, J.; Zhao, J.; Zhao, S.; Li, J.; Liu, J.; Jiang, L.; and Guan, H. 2025. FlexQuant: A Flexible and Efficient Dynamic Precision Switching Framework for LLM Quantization. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 4152–4161. Association for Computational Linguistics.

Nallapati, R.; Zhou, B.; dos Santos, C.; Gülçehre, Ç.; and Xiang, B. 2016. Abstractive Text Summarization Using Sequence-to-Sequence RNNs and Beyond. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, 280–290. Association for Computational Linguistics.

Papineni, K.; Roukos, S.; Ward, T.; and Zhu, W.-J. 2002. BLEU: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 311–318. Association for Computational Linguistics.

Post, M. 2018. A Call for Clarity in Reporting BLEU Scores. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, 186–191. Association for Computational Linguistics.

Sakaguchi, K.; Le Bras, R.; Bhagavatula, C.; and Choi, Y. 2020. WinoGrande: An Adversarial Winograd Schema Challenge at Scale. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(5): 8732–8740.

Sun, M.; Liu, Z.; Bair, A.; and Kolter, J. Z. 2024. A Simple and Effective Pruning Approach for Large Language Models. In *International Conference on Learning Representations*.

Sun, Y.; Liu, R.; Bai, H.; Bao, H.; Zhao, K.; Li, Y.; Hu, J.; Yu, X.; Hou, L.; Yuan, C.; Jiang, X.; Liu, W.; and Yao, J. 2025. FlatQuant: Flatness Matters for LLM Quantization. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 57587–57613. PMLR.

Thakkar, V.; Ramani, P.; Cecka, C.; Shivam, A.; Lu, H.; Yan, E.; Kosaian, J.; Hoemmen, M.; Wu, H.; Kerr, A.; Nicely, M.; Merrill, D.; Blasig, D.; Atluri, A.; Qiao, F.; Majcher, P.; Springer, P.; Hohnerbach, M.; Wang, J.; and Gupta, M. 2023. CUTLASS. https://github.com/NVIDIA/cutlass/tree/v4.5.0. Version 4.5.0.

Touvron, H.; et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*.

Williams, S.; Waterman, A.; and Patterson, D. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4): 65–76.

Xiao, G.; Lin, J.; Seznec, M.; Wu, H.; Demouth, J.; and Han, S. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, 38087–38099. PMLR.

Xu, P.; Shao, W.; Chen, M.; Tang, S.; Zhang, K.; Gao, P.; An, F.; Qiao, Y.; and Luo, P. 2024. BESA: Pruning Large Language Models with Blockwise Parameter-Efficient Sparsity Allocation. In *International Conference on Learning Representations*.

Yin, L.; Wu, Y.; Zhang, Z.; Hsieh, C.-Y.; Wang, Y.; Jia, Y.; Li, G.; Jaiswal, A. K.; Pechenizkiy, M.; Liang, Y.; Bendersky, M.; Wang, Z.; and Liu, S. 2024. Outlier Weighed Layerwise Sparsity (OWL): A Missing Secret Sauce for Pruning LLMs to High Sparsity. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 57101–57115. PMLR.

Zhao, J.; Wan, B.; Wu, C.; Peng, Y.; and Lin, H. 2024. LLM-PQ: Serving LLM on Heterogeneous Clusters with Phase-Aware Partition and Adaptive Quantization. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 460–462. Association for Computing Machinery.